컴퓨터시스템프로그래밍

Quiz 1001

Dynamic Memory Allocation: Advanced Concepts

이름	
학번	
범위	08번 31~32쪽 · 09번 1~36쪽
문항	총 15문항 — OX 5 / 단답·서술 6 / 설계·추론 4

안내
- 각 문항의 요구 산출물을 빠짐없이 제시하세요.
- OX 문항은 정답만 쓰지 말고, 거짓인 경우 반드시 올바르게 고치세요.
- 필요한 경우 그림·화살표·표를 답안 공간에 추가해도 됩니다.
- 시간과 배점은 별도로 제시하지 않습니다.

답안은 개념의 관계와 근거가 드러나도록 작성하세요.

A. OX

각 진술의 O/X를 쓰고, X인 경우 올바르게 고치시오.

1. 병합 정책

"Deferred coalescing은 **free가 호출될 때마다** 인접 빈 블록을 즉시 병합하는 정책이다."

2. 빈 블록의 포인터

"Explicit free list의 `next`·`prev`는 free 상태 블록의 사용되지 않는 payload에 저장할 수 있으므로, 그 사실만으로 allocated block의 internal fragmentation이 늘어나지는 않는다."

3. 논리 순서와 주소 순서

"Explicit free list에서 `next`를 따라가는 순서는 항상 실제 블록 주소의 오름차순과 같다."

4. Seglist의 다음 단계

"Seglist에서 요청 크기에 맞는 class에 적절한 free block이 없으면, 더 큰 size class를 시도하고, 그래도 없을 때 OS에 추가 heap을 요청할 수 있다."

5. 양수 count와 도달 가능성

"어떤 block의 reference count가 양수이면, 그 block은 반드시 root에서 도달 가능하다." 반례 구조를 짧게 함께 쓰시오.

B. 단답·서술

핵심 용어만 나열하지 말고, 이유와 관계를 문장으로 설명하시오.

6. Implicit와 explicit의 탐색 비용

(1) implicit list의 최악 할당 시간이 선형인 이유, (2) explicit list가 메모리 대부분이 사용 중일 때 더 유리한 이유를 각각 설명하시오.

7. Explicit free list 삽입 정책

LIFO와 address-ordered 삽입 정책을 비교하여, (1) 새 free block의 삽입 비용, (2) 인접 블록을 찾아 병합하기 쉬운 정도를 설명하시오.

B. 단답·서술 (계속)

문제에서 header·정렬을 무시하라고 하면 논리 크기만 사용하시오.

8. Seglist에서 분할 뒤의 잔여 블록

size class가 **1~16, 17~32, 33~64**이고, 17~32 class에서 크기 24인 첫 블록을 찾아 논리 크기 20 요청에 사용한다고 하자. header·정렬은 무시한다. 할당 뒤 남는 블록의 크기와 그 블록이 들어갈 free list class를 쓰시오.

9. Garbage graph의 root와 garbage

힙 밖 root `r`이 A를 가리키고 `A → B → C`이다. 별도로 `D → E → D`가 있지만 D/E로 들어오는 root 경로는 없다. (1) root node를 정의하고, (2) A~E 중 회수 후보를 쓰고 이유를 설명하시오.

10. GC가 필요한 포인터 가정

강의에서 제시한 pointer 관련 가정 세 가지를 쓰고, 포인터를 정수로 캐스팅해 숨겼다가 다시 포인터로 쓰는 것이 GC를 어렵게 만드는 이유를 설명하시오.

B. 단답·서술 (계속)

포인터 대입 뒤에는 외부 참조와 구조체 내부 참조를 모두 세시오.

11. Reference count의 연쇄 해제

A/B는 각각 노드 10/20이다. 위 코드에서 두 번째 줄 뒤, `a=b` 뒤, `b=NULL` 뒤의 (A, B) reference count를 쓰고, 마지막 대입 후 해제되는 노드를 순서대로 설명하시오.

```
node_t *a = gen_node(10, NULL); // a → A
node_t *b = gen_node(20, a);    // b → B, B.next → A
a = b;
b = NULL;
a = NULL;
```

C. 설계·추론

하나의 정책만 단정하지 말고, 선택의 이점과 비용을 함께 드러내시오.

12. 병합 시점 설계

짧은 객체를 대량으로 `free`하는 구간이 매우 자주 있고, 그 직후에는 큰 할당 요청이 거의 없다. 다만 일정 시간이 지나면 큰 블록이 필요한 요청이 들어올 수 있다. immediate/deferred coalescing 중 하나를 선택하고, 선택 이유와 반대 정책의 비용을 각각 설명하시오.

C. 설계·추론 (계속)

구조를 그려도 좋으며, 포인터의 방향과 NULL 여부를 명확히 표시하시오.

13. LIFO 삽입의 링크 갱신

기존 free list가 `head → X ⇔ Y`이고 `X.prev = NULL`이다. 새로 free된 `F`를 LIFO 정책으로 머리에 삽입할 때, `head`, `F.next`, `F.prev`, `X.prev`에 저장할 값을 쓰고, 유지해야 할 이중 연결 리스트 불변식 한 가지를 적으시오.

C. 설계·추론 (계속)

요청 크기보다 작은 block은 해당 요청을 만족시킬 수 없다.

14. Seglist 할당 경로 설계

다음 free list 상태에서 논리 크기 24를 할당한다. header·정렬은 무시한다. (1) 첫 탐색 class, (2) 그 class에서 실패하는 이유, (3) 다음에 사용할 block, (4) 분할 뒤 잔여 block의 class를 순서대로 쓰시오.

class	free block (리스트 순서)
1~16	8, 12
17~32	18
33~64	40

C. 설계·추론 (계속)

reference count와 root에서 출발한 도달성 판단은 같은 질문에 다른 정보를 제공한다.

15. Cycle garbage에 대한 회수 판단

유일한 외부 root `r`은 `P`를 가리킨다. `P → Q`, `Q → P`이고, 별도 cycle `U → V`, `V → U`에는 root에서 들어오는 경로가 없다. 다음을 답하시오.

(1) reference counting만 사용할 때 P/Q와 U/V 각각이 왜 해제되거나 남는가?

(2) root에서 그래프를 따라 도달성을 표시하는 회수 방식이라면 어느 노드를 보존하고 어느 노드를 garbage로 표시하는가?

(3) 이 사례가 `count > 0` ≠ `reachable`을 반박하는 이유는 무엇인가?